Enrico Bravi

enrico.bravi@polito.it

Module E – OS Security

E2 – OS Security

Politecnico di Torino

Department of Control and Computer Engineering

TORSEC

Computer and Network Security Group

Acknowledgments

This lecture deck *adapts* and *reorganizes* material originally developed by Prachi Gulihar. The presentation includes material from different sources. Portions of the content are reused under the Creative Commons BY-NC license (see license slide). All modifications, additions, re-organization, and any errors are the responsibility of the course instructor.

License & Disclaimer

License Information

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.
- This material is used and adapted within the course “Cybersecurity and National Defense”.

Security

- in information technology, security means protecting digital information and IT systems from threats, whether those threats are deliberate or accidental, and whether they come from within or outside the organization
- this protection relies on identifying risks, blocking potential attacks, and responding to incidents, using several security protocols, dedicated software, and IT support services
- security is all about defending your valuable digital assets

Least Privilege Principle

- the ultimate security aim is to restrict a process strictly to the actions it needs to perform
- this principle is a cornerstone of cybersecurity, stating that users must have only the permissions essential for their specific roles
- it extends beyond users to include computer processes as well
- every system element or process should operate with the minimum level of access required to fulfill its goals

Least Privilege Principle - Advantages

- system stability: restricting the range of changes code can make simplifies testing its behavior and how it interacts with other software
- system security: limiting the system-wide actions of code prevents vulnerabilities in one program from compromising the entire system
- simplified deployment process: applications that require fewer permissions are generally easier to integrate and manage within complex environments

Access Control

- access control in information security permits an organization's leadership to specify and regulate which users can reach particular systems or resources, as well as define the actions those users are authorized to perform
- access control manages whether a subject (an active entity like a user or process) is allowed to interact with an object (a passive entity such as a file or system)
- these permissions are typically established through defined rules or rights (read, write, execute, list, modify, and delete) and are enforced using different security strategies (administrative, technical, and physical safeguards)

Access Control – Objects (I)

- typical resources requiring security controls include:
  - system memory
  - files stored on disk
  - running processes
  - file directories
  - execution stacks
  - critical instructions, particularly those with elevated privileges
  - user credentials and authentication systems

Access Control – Objects (II)

- these objectives work together for securing resources effectively
  - verify every access attempt
    - even if a user has been granted permission before, it doesn't mean they should have unlimited or permanent access to the resource
  - apply the least privilege principle
    - users should only be granted access to the minimum set of resources necessary to complete their tasks

Access Control

- access control mechanisms regulate and approve requests from various entities (such as users, applications, or services) to carry out actions (like reading, writing, or executing) on resources (including files, network sockets, or databases)
- the foundation of access control rests on two key elements
  - a security framework that defines the rules and policies governing access permissions
  - a reference monitor, which acts as the enforcement agent within the system, ensuring that these access rules are strictly applied

Protection System (I)

- a protection system is composed of a protection state, which outlines the actions that system users (subjects) are permitted to carry out on system resources (objects), along with a collection of operations that allow changes to this state
- it facilitates the establishment and administration of the protection state
- the protection state itself includes the particular users, the specific resources, and the authorized actions those users can execute on those resources
- additionally, the protection system specifies operations that permit modifications to the protection state, ensuring dynamic control over access rights.

Protection System (II)

- the access matrix serves as a framework to outline a process's protection domain
- a protection domain defines which resources (or objects) a process can interact with, along with the specific actions it is permitted to perform on those resources
- by reviewing the rows within the access matrix, one can identify all the operations a subject is allowed to execute on various system resources

	File 1	File 2	File 3	Process 1	Process 2
Process 1	Read	Read, Write	Read, Write	Read	-
Process 2	-	Read	Read, Write	-	Read

Mandatory Protection System (I)

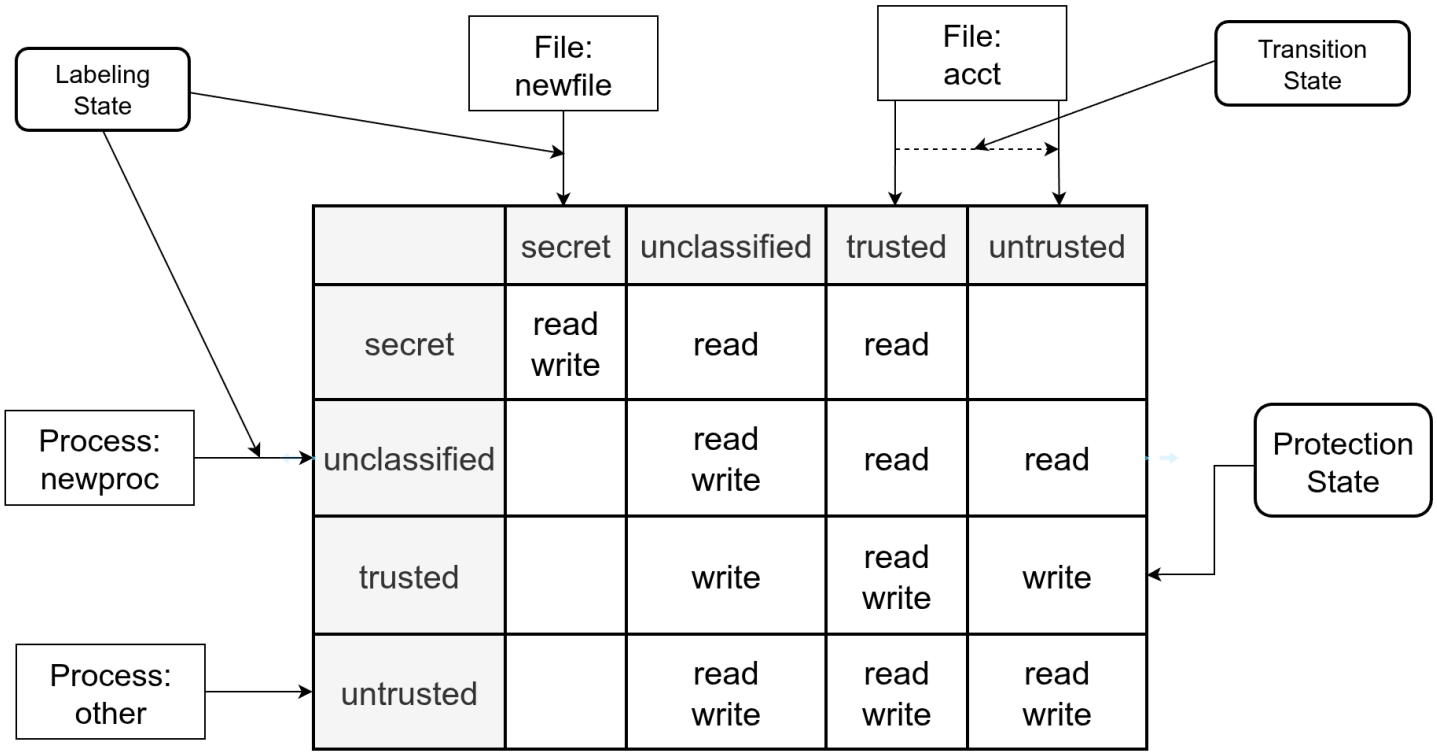
- the access matrix model introduces a challenge for maintaining secure systems: processes that aren't trusted might interfere with the protection mechanisms
- through operations that alter the protection state, untrusted user processes can change the access matrix by adding new subjects, objects, or permissions within its cells
- for example, if Process 1 owns File 1, it can grant other processes permissions such as read, write, or even ownership rights over File 1
- a protection framework that allows untrusted processes to alter the protection state is known as a discretionary access control (DAC) system

Mandatory Protection System (II)

- a mandatory protection system is one that only authorized administrators can modify, through trusted software, having the follow state representations
  - the mandatory protection state defines a state where both subjects and objects are assigned specific labels determining the possible actions that subjects can perform on objects
  - the labeling state as a mapping mechanism, linking processes and system resource objects to their respective labels
  - the transition state is the authorized procedures by which processes and system resource objects may change label

Mandatory Protection System (III)

- the system's protection status is characterized by immutable labels
- this fixed labeling, along with defined state transitions, facilitates the control of labels for subjects and objects



	secret	unclassified	trusted	untrusted
secret	read write	read	read	
unclassified		read write	read	read
trusted		write	read write	write
untrusted		read write	read write	read write

Mandatory Access Control (MAC)

- in a MAC framework, control over who can access certain data is determined by the system itself, rather than by individual users
- this model relies on security labels where each user (subject) is assigned a security authorizations (such as confidential, secret, or top secret), while each data item (object) is tagged with a corresponding security classification
- these labels, which link authorizations and classifications to their respective subjects and objects, govern access control
- MAC is predominantly implemented in scenarios where protecting sensitive information is critical, like military organizations

Discretionary Access Control (DAC)

- DAC allows the owner of a resource to determine who can access it
- It is called “discretionary” because the owner has the authority to grant or restrict access at their own judgment
- this access control model is at the base of the most popular operating systems, including Windows, Linux, macOS, and many Unix variants
- in these systems, when you create a file, you have the power to set permissions for other users and the OS then enforces these permissions, deciding access based on the rules you established

Role Based Access Control (RBAC)

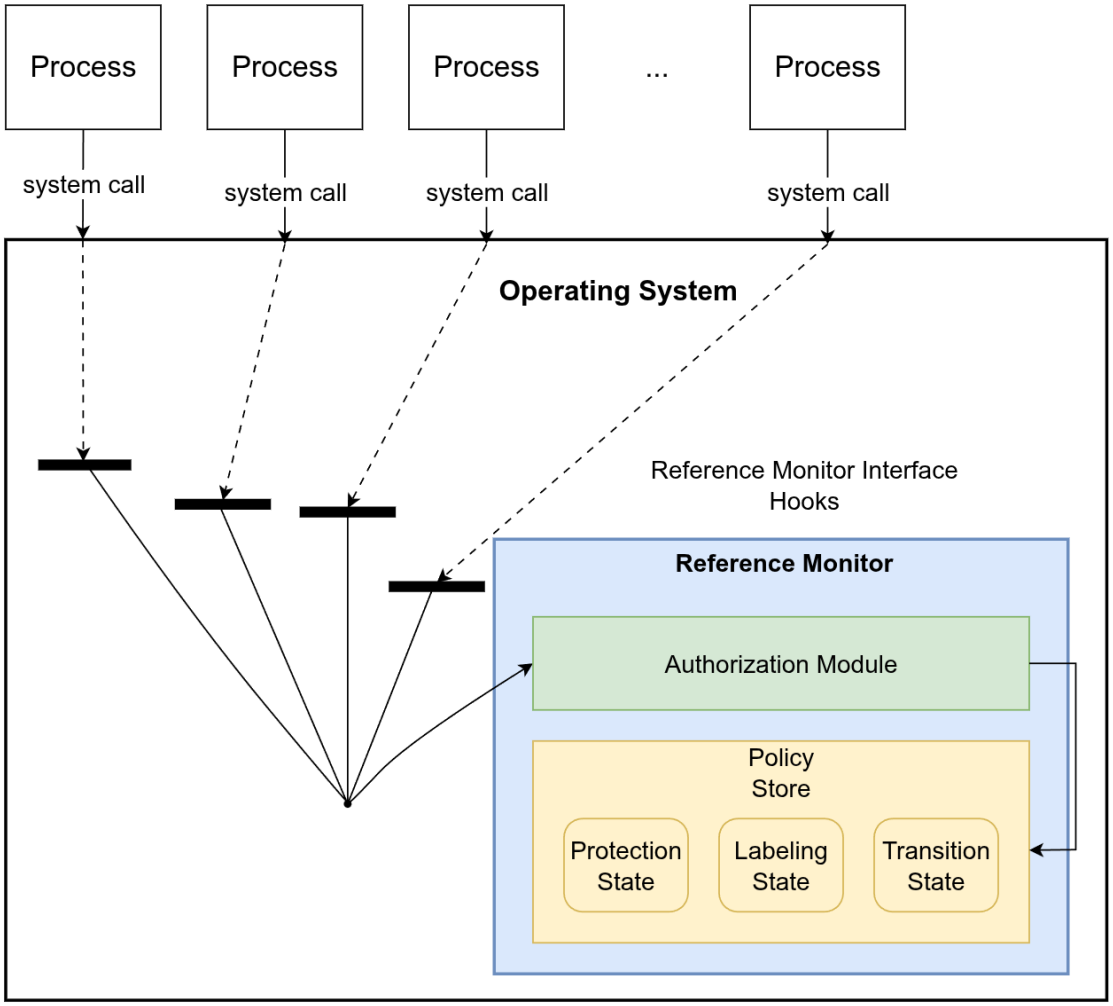
- RBAC is a strategy for managing system access by assigning permissions based on users' roles
- these roles are assigned according to each individual's competency, authority level, and responsibilities within the organization
- It allows users to perform a large spectrum of authorized activities by adjusting their permissions dynamically, reflecting their functions, relationships, and any applicable restrictions
- roles can be easily created, modified, or retired as organizational needs change, removing the need to update each user's access rights one by one

Reference Monitors (I)

- the reference monitor serves as the foundational mechanism for enforcing access control
- it consists of three key elements:
  - the interface through which access requests are made
  - the authorization module that evaluates these requests
  - the policy store that holds the rules governing access
- after an access request, the reference monitor interface triggers specific hooks that call upon the authorization module
- this module then queries the policy store to determine whether access should be granted, including handling authorization checks, labeling decisions, and transitions between labels based on the current system state.

Reference Monitors (II)

- this diagram illustrates a simplified model of a reference monitor
- it processes incoming requests and delivers a clear decision
- the decision reflects whether the request complies with the access control rules defined by the reference monitor



Reference Monitor Interface

- the reference monitor interface specifies the points at which the protection system interacts with the reference monitor
- it guarantees that every operation involving security-sensitive actions are authorized by the access control mechanism
- a security-sensitive action refers to any operation performed on a specific object (such as a file, socket) that could potentially breach the system's security policies if executed without proper authorization
- for instance, an operating system manages file access operations to prevent one user from reading another user's confidential information (like a private key) unless explicitly permitted by the system's security controls

Authorization Module

- at the heart of the reference monitor lies the authorization module, which processes inputs from the interface, such as the identity of the process, references to objects, and the name of the system call, and transforms these into a query directed at the policy repository
- a key challenge for this module is accurately translating the process identity into a subject label, converting object references into corresponding object labels, and identifying the specific operations that require authorization, noting that multiple operations might be involved per interface
- while the protection system defines the available labels and operations, it is the authorization module's responsibility to create an effective method for mapping these elements to generate the correct query for enforcement

Policy Store (I)

- the policy store serves as a centralized repository managing protection states, labeling statuses, and transition conditions
- when the authorization module sends a query, the policy store evaluates it and provides a response
- authorization queries typically include a combination of {subject_label, object_label, operation_set} and yield a simple yes/no authorization decision
- labeling queries involve {subject_label, resource}, where the subject's label and certain system resource attributes influence the label assigned to the resource

Policy Store (II)

- resources can be dynamic entities like processes or static objects such as files
- additionally, some systems perform queries to validate state transitions, ensuring secure changes in system status.

Filesystem Security

- a file system serves as a structured approach to storing and managing computer files along with their data, ensuring quick and easy retrieval
- these systems are implemented across various storage devices, including hard drives, USB flash drives, CDs, DVDs, and other digital storage media
- storage hardware typically consists of numerous fixed-size units, often referred to as sectors or blocks, which the file system organizes into coherent files and folders
- additionally, the file system maintains an index of the storage medium, enabling it to accurately locate and identify each file's position and content

Types of Filesystem

- Disk-based file systems include FAT (File Allocation Table), NTFS, HFS (Hierarchical File System), ext2, ext3, ISO9660, and UDF
- Windows platforms predominantly utilize FAT variants (FAT12, FAT16, FAT32) and especially NTFS for managing files
  - the FAT system remains the go-to format for floppy disks and continues to be in use today
- Mac OS relies on HFS, while Linux distributions commonly employ ext2 and ext3 file systems
- optical storage media such as CDs and DVDs use ISO9660 and UDF file systems respectively

Filesystem Security

- the file system plays a vital role in maintaining data integrity, primarily by enforcing access controls
- operations like editing or deleting files are regulated through mechanisms such as access control lists (ACLs) or capabilities
  - capabilities offer stronger security and are commonly implemented in operating systems managing file systems like NTFS and ext3
- additionally, backup and recovery solutions serve as a secondary layer of defense to protect data

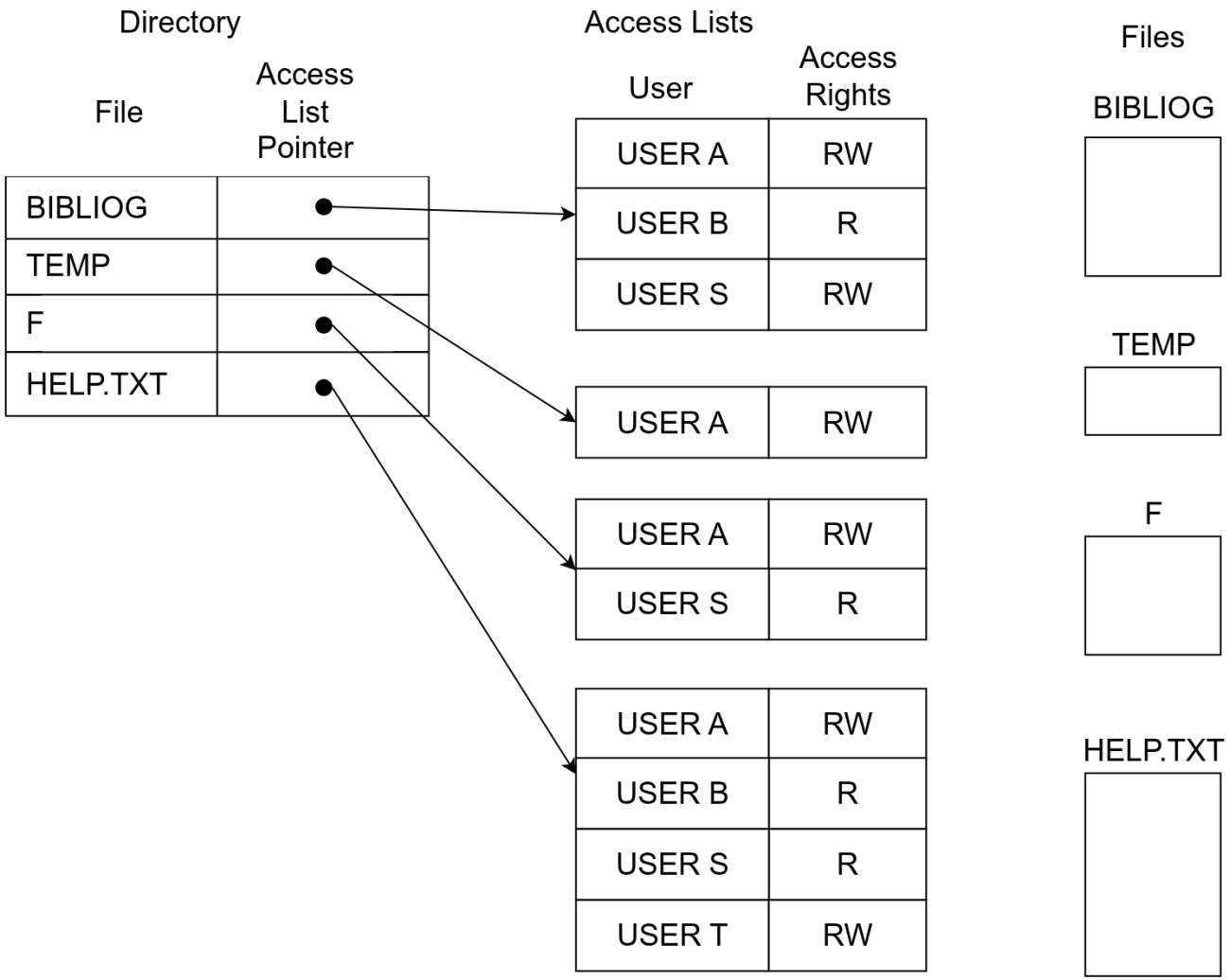
Access Control - Files

- controlling access is a critical component of securing file systems
  - the system must restrict file access strictly to authorized users
- most leading file systems implement Access Control Lists (ACLs) or capability-based controls to guard against unauthorized or harmful actions
  - user permissions typically determine whether they can read, modify, or execute a file
- in certain file system designs, only specific users have the authority to change ACL settings or even detect the presence of a file
  - limiting user access reduces potential risks and helps maintain the system's integrity more effectively

Access Control Lists (I)

- each object is associated with a unique ACL
- an ACL details every subject permitted to access the object and specifies their level of access
  - for every object, there is a single access control list; meanwhile, a directory is maintained for each subject
- the ACL of a file encapsulates its access control details
  - it includes all non-empty entries that would appear in the file's column within the Access Control Matrix (ACM)

Access Control Lists (II)



Access Control Matrix (I)

- the access control matrix serves as a security framework designed to efficiently manage:
  - the permissions users have for different files
  - the control details governing file access
- this matrix is organized as a table where each row corresponds to a subject (such as a user), each column corresponds to an object (like a file), and each cell specifies the access rights that the subject holds over that object

Access Control Matrix (II)

Access Control Matrix

	Subject	File 1	File 2	**File 3**	File 4
	Larry	Read	Read, write	Read	Read, write
Capability	Curly	Full control	No access	Full control	Read
	Mo	Read, write	No access	Read	Full control
	Bob	Full control	Full control	No access	No access
				ACL	

Capability = row in matrix
ACL = column in matrix

Operating System (I)

- an operating system (OS) is the core software loaded into a computer during startup by a boot program and it coordinates all other software running on the machine, commonly known as applications or application programs
- a critical concern for the OS is protection and security: it must guarantee that every resource or object is accessed appropriately and only by authorized processes belonging to permitted users
- designing an OS involves the complex task of developing a security framework robust enough to prevent any future software from bypassing these protections

Operating System (II)

- a secure OS is one that rigorously enforces access controls in line with the reference monitor principle which sets the foundational requirements for any system aiming to provide mandatory security protections, anchored by three fundamental guarantees
- Complete Mediation: the system enforce and evaluates every security-sensitive operation
- Tamperproof: the access control framework and its security components are protected against unauthorized modifications by any untrusted software
- Verifiable: the enforcement mechanism is deliberately designed to be sufficiently compact, enabling comprehensive analysis and testing to confirm that the system reliably achieves its security objectives

OS Security Methods

- enforcing separation among various components within the same system is fundamental
- this separation can be achieved through several approaches:
  - physical isolation: assigning each module or process its own dedicated hardware terminal or device
  - temporal isolation: running processes at separate times to prevent any concurrent execution
  - logical isolation: the OS provides an abstracted environment, offering users isolated logical workspaces without exposing internal system details
  - cryptographic isolation: employing encryption methods to protect and hide sensitive data from unauthorized access

Security Kernel

- it acts as the core component responsible for enforcing security protocols across the entire operating system
- serves as the critical interface connecting hardware, the OS, and other system components to maintain security
- implementing a security kernel can sometimes impact system speed or increase the overall file size
- defined as the combination of hardware and software elements that represent the reference monitor concept
- the concept of the security kernel was first brought to life with a prototype developed by MITRE in 1974

OS Security Example – UNIX (I)

- UNIX is a multiuser operating system originally created by Dennis Ritchie and Ken Thompson at AT&T Bell Labs
- it incorporates several security mechanisms inspired by the Multics system, including encrypted password storage, the use of protection rings, and access control lists
- the UNIX environment operates with a core kernel and numerous processes, each executing its own program
- a protection ring acts as a security boundary, separating the kernel from user-level processes
- every process is allocated a unique address space, which specifies the range of memory locations it can access

OS Security Example – UNIX (II)

- modern UNIX operating systems manage address spaces by defining which memory pages a process can access
- in UNIX, every persistent system resource, ranging from secondary storage and input/output devices to network interfaces and interprocess communication channels, is represented as a file
- each UNIX process carries a user-based identity, which governs its permissions and restricts file access accordingly
- the primary goal of UNIX security is to safeguard users from one another while also protecting the system's trusted computing base (TCB) from any user threats

OS Security Example – UNIX (III)

- at its core, the UNIX TCB includes the operating system kernel along with several key processes that operate under the privileged “root” or superuser account
- these root-level processes handle critical functions such as system startup, verifying user identities, system management, and providing network-related services
- both the kernel and these privileged processes possess unrestricted access to all system resources
- in contrast, all other processes run with permissions limited by the specific user account they belong to, restricting their system access accordingly

OS Security Example – Windows (I)

- Microsoft Windows' roots trace back to MS-DOS, the foundational operating system launched for IBM personal computers in 1981
- the security framework in Windows 2000 mirrors UNIX's DAC model, managing protection states, object labels, and transitions between protection domains
- key distinctions between the two systems lie in their adaptability and expressive capabilities: Windows offers greater extensibility and supports a broader range of security policies compared to UNIX

OS Security Example – Windows (II)

- evaluating Windows' security framework against the ideal standards of a robust protection system reveals that its enhanced flexibility and richer feature set introduce additional challenges in maintaining security
- the Windows security model differs from UNIX primarily due to its broader range of objects and operations, along with greater adaptability in how these are assigned to users and processes
- subjects in Windows are as in UNIX; each process is linked to a token that encapsulates the process's identity
- While a Windows identity corresponds to a single user, the process token for that user can include a diverse mix of permissions and rights, offering multiple combination

OS Security Example – Windows (III)

- unlike UNIX, Windows manages a variety of object types beyond just files
- software programs can introduce new object categories and register them within the Active Directory, which serves as the system's hierarchical namespace for all recognized objects
- from the standpoint of access control, each object type is characterized by the specific operations it supports
- a significant distinction between Windows and UNIX security models is that Windows employs flexible ACLs, offering more granular permissions compared to UNIX's simpler mode bits system.

OS Security Example – Android (I)

- Android is a mobile platform created by Google, built upon a customized Linux kernel and a collection of open-source components
- it is primarily tailored for devices with touchscreens, including smartphones and tablets
- the user interface of Android emphasizes intuitive, direct interaction relying on touch gestures that mimic physical actions, such as swiping, tapping, pinching, and spreading fingers, to control elements on the screen, complemented by an on-screen virtual keyboard

OS Security Example – Android (II)

- Android apps operate within a secure sandbox environment, which isolates them from the rest of the system and they can only access other system resources if the user explicitly grants permission during installation
- the platform leverages Security-Enhanced Linux (SELinux) to enforce strict access control policies, implementing MAC on running processes
- starting with Android 2.2, the Android Device Administration API offers system-level device management capabilities, enabling enhanced control and security features

OS Security Example – Android (III)

- Android's security framework is built upon the robust Linux kernel, which serves as the foundation for its mobile computing environment
- key security capabilities provided by the Linux kernel include:
  - a permissions system centered around user roles, ensuring controlled access
  - isolation of processes to prevent unauthorized interactions
  - a flexible and secure inter-process communication (IPC) mechanism
  - the option to strip away unnecessary or vulnerable kernel components to enhance security

OS Security Example – Android (IV)

- Android, built on a multiuser Linux foundation, prioritizes strict separation of user environments to safeguard individual resources
- the core Linux security approach focuses on preventing interference between users by:
  - blocking user A from accessing user B's files
  - preventing user A from reducing user B's memory allocation
  - restricting user A from monopolizing user B's CPU time
  - ensuring user A cannot abuse shared hardware like telephony, GPS, or Bluetooth devices assigned to user B

OS Security Example – Android (V)

- System Partition and Safe Mode
  - the system partition hosts the Android kernel along with the core operating system components such as libraries, the application runtime, framework, and built-in apps, which is locked to read-only mode to protect its integrity
  - when a device boots into Safe Mode, third-party apps do not start automatically, and the users can manually launch these apps if needed

OS Security Example – Android (VI)

- Filesystem Permissions
  - Android employs a UNIX-like permission system to safeguard user data by preventing one app from accessing or modifying another app's files
  - each application operates under a unique user identity, ensuring isolation
  - unless a developer explicitly allows file sharing, an app's data remains private and inaccessible to others

OS Security Example – Android (VII)

- Security-Enhanced Linux (SELinux)
  - Android integrates SELinux to enforce strict access control policies, implementing MAC to regulate process permissions and enhance system security
- Verified Boot Mechanism
  - starting with Android 6.0, devices support verified boot alongside device-mapper-verity, ensuring the software's integrity from the hardware root of trust through to the system partition
  - during the startup sequence, each boot phase cryptographically validates the next stage's authenticity and integrity before allowing it to run
  - from Android 7.0 onward, verified boot is enforced rigorously, preventing devices with compromised software from completing the boot process

References

- Charles P. Pfleeger, Shari Lawrence Pfleeger, Analysing Computer Security, 4th Edition, ISBN: 9780132390774, Prentice Hall
- https://source.android.com/security/overview/kernel-security
- Edmund L. Burke, Synthesis of a software security system, Proceedings of the 1974 annual ACM conference